Planification de la production d'électricité - Antoine Ackermann

In [1]:
```
import pyomo.environ as pyo
import pandas as pd
import logging
import copy

logging.getLogger('pyomo.core').setLevel(logging.ERROR) # éviter les messages d'erreurs

optimizer = pyo.SolverFactory("gurobi")
```

Note pour tout le document : par souci de clarté du code et de longueur du notebook, le modèle propre à chaque question est implémenté au sein d'une fonction *build_model()* stockée chacune dans un fichier .py externe joint.

1. Modèle de base des centrales thermiques

1.1 Planification journalière du parc thermique

On définit les variables suivantes :

- $x_{p,t}$ = production de la technologie $p \in \{A, B, C\}$ à la période $t \in T$
- $n_{p,t}$ = nombres entiers ($n_{p,t} \in \mathbb{Z}_+$) de centrales de la technologie p allumée à la période t

Le problème d'optimisation linéaire se définit comme :

$$\min \sum_{t \in T} \sum_{p \in A,B,C} C_{MWh,t} \cdot x_{p,t} \cdot \Delta_t$$

s.c. :

- $\sum_{p \in \{A,B,C\}} x_t \geq D_t$ (satisfaction de la demande)
- $n_{p,t} \cdot P_{min,p} \leq x_{p,t} \leq n_{p,t} \cdot P_{max,p} \; \forall p, t$ (la puissance associée à la production de chaque technologie doit rester dans les limites techniques de fonctionnement dictées par les caractéristiques des centrales)
- $0 \leq n_{p,t} \leq N_p \; \forall p, t$ (le nombre de centrale en fonctionnement ne peut dépasser le nombre de centrales disponibles dans le parc)

Le problème serait irréalisable dans le cas où la demande ne pourrait être satisfaite par le parc existant sur une période donnée, c'est-à-dire : $\exists t \in T, d_t > \sum_{p \in A,B,C} P_{max,p} N_p$.

Dans les conditions actuelles, le problème est borné. Il ne le serait pas si on supprimait les conditions sur les bornes sur les P_{max}, p et si on introduisait un coût marginal négatif sur une technlogie : car on pourrait alors produire une puissance arbitrairement grande avec coût total allant vers $-\infty$.

S'agissant de la contrainte sur la demande, on fait le choix d'une inégalité $\geq$ et non d'une égalité stricte car la contrainte sur les P_{min} rend impossible l'atteinte de certaines valeurs.

Le problème est *décomposable* en problèmes indépendants car les variables de chaque période sont indépendantes des variables des périodes suivantes ou précédentes et l'objectif est une somme de contributions ne dépendant pas de t. Ainsi, la somme des coûts minimisés de la production sur chaque période est égale à la minimisation de la somme des coûts de chaque période. Cela changera lorsque nous introduirons des coûts de démarrage (qui dépendent donc de l'état des centrales à la période précédente) dans la question 2.2.

1.2. Relaxation continue

La relaxation continue consiste ici à considérer que $n_{p,t} \in \mathbb{R}$, c'est-à-dire qu'il est possible d'allumer des "fractions de centrales". La contrainte $n_{p,t} \cdot P_{min,p} \leq x_{p,t}$ devient donc inopérante, car $n_{p,t}$ peut devenir aussi petit que possible. Comme toutes les valeurs de $x_{p,t}$ sont accessibles dans ce cas, la contrainte sur la demande peut devenir une égalité.

Une solution est alors de trier les centrales par coût marginal croissant et remplir la demande jusqu'à saturation de chaque catégorie dans l'ordre des coûts : $c_b < c_a < c_c$.

1.3. Solutions basiques

1. Modèle réduit

Dans le cas de la relaxation continue, la contrainte $n_{p,t} \cdot P_{min,p} \leq x_{p,t}$ est supprimée. Le problème devient donc :

$$\min \quad c_a x_a + c_b x_b + c_c x_c \quad \text{s.c} \quad \begin{cases} x_a + x_b + x_c = D_t \\ 0 \leq x_p \leq N_p P_{max,p} \quad \forall p \in \{A, B, C\} \end{cases}$$

On a bien 3 variables reliées par une contrainte sous forme d'inégalité.

2. Modèle en forme standard

En passant en forme standard, le problème sur la période 2 s'écrit :

$$(P2_{standard}) : \min \sum_p c_p x_p \quad \text{s.c.} \quad \begin{cases} x_A + x_B + x_C = D_2 \\ x_p + s_p = U_p \\ x_p \geq 0 \\ s_p \geq 0 \end{cases} \quad \forall p \in \{A, B, C\}$$

où les s_p correspondent aux variables d'écart et où $U_p = P_{max} N_p$.

3. Modèle en forme duale

On note λ la variable duale associée à la contrainte sur la satisfaction de la demande et μ_p les variables duales associées aux contraintes sur la production des centrales. Le problème dual devient :

$$(P2_{dual}) : \max \quad \lambda D_2 + \sum_p U_p \mu_P \quad \text{s.c.} \quad \begin{cases} \lambda + \mu_A \leq c_A \\ \lambda + \mu_B \leq c_B \\ \lambda + \mu_C \leq c_C \\ \mu_A \leq 0, \mu_B \leq 0, \mu_C \leq 0 \end{cases}$$

4. Uniquement des centrales A

- La solution est faisable ssi $0 \leq D_2 \leq U_A$. On sait que $U_A = N_A P_{max,A} = 24000$ MW et la demande pour la période 2 est de 30000 MW, donc la **solution n'est pas faisable**.
- La solution sur la période 2 est l'intersection de $x_B = 0, x_C = 0, x_A = D_2$ soit 3 contraintes linéairement indépendantes dans $\mathbb{R}^3$: il s'agit d'un sommet du polyèdre formé par l'intersection des demi-plans formés par les contraintes, **c'est donc une solution basique**.
- Une solution est dégénérée si il y a plus de contraintes actives que de variables. C'est le cas lorsque $D_2 = 0$ et $D_2 = U_A$: la contrainte sur la demande est active et les trois variables de production sont à leur bornes, c'est-à-dire que 4 contraintes sont actives.
- Tant que $D_2 > 0$ et qu'il existe de la capacité B disponible, la solution « tout A » n'est pas optimale, puisque $c_B < c_A$: on peut remplacer une partie de x_A par x_B et réduire le coût.

5. Solution Merit-Order (MO)

Pour la période 2, la solution MO est : $x_B = U_B = 17500, x_A = D_2 - x_B = 12500, x_C = 0$. On a donc : $(x_A, x_B, x_C, s_A, s_B, s_C) = (12500, 17500, 0, 11500, 0, 20000)$.

$A = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}$. Les variables non nulles d'une solution basique doivent former une base. Pour cela, la sous-matrice associée doit être inversible. Ici, la sous-matrice associée à la base $\beta = (1, 2, 4, 6)$ est $A_\beta = \begin{pmatrix} 1 & 1 & 0 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}$ qui est inversible car les lignes sont linéairement indépendantes. C'est donc une **solution basique**.

Il y a 3 contraintes actives pour 3 variables, ce n'est donc **pas une solution dégénérée**.

D'après les conditions de complémentarité, les contraintes duales associées aux variables primales non-nulles sont saturées. On a donc le système suivant :

$$\begin{cases} c_A - \lambda - \mu_A = 0 \\ c_B - \lambda - \mu_B = 0 \\ \mu_A = 0 \\ \mu_C = 0 \end{cases} \Rightarrow \begin{cases} \lambda = c_A \\ \mu_A = 0 \\ \mu_B = c_B - c_A \\ \mu_c = 0 \end{cases}$$

On vérifie les contraintes non-actives : $\lambda + \mu_C = c_A \leq c_C$ et $\mu_B = c_B - c_A \leq 0$. Toutes les conditions KKT sont vérifiées : la solution (MO) sur la période **est bien optimale** et la solution duale associée est : $(\lambda, \mu_A, \mu_B, \mu_C) = (c_A, 0, c_B - c_A, 0)$.

1.4 Analyse de sensibilité

Rappel de la solution primale : $x_A = D_2 - U_B, x_B = U_B, x_c = 0$.

1. Pour la période 1, la demande est servie uniquement par B, sans saturer U_B : la base reste donc la même (les mêmes contraintes restent actives). Donc par dualité, le coût marginal d'une augmentation de 1 MW de la demande sur une période est la valeur de la variable duale associée à la contrainte de demande de cette période : la variable duale et ici le *shadow price*, c'est-à-dire le coût additionel induit en resserrant la contrainte d'une unité infinitésimale.La période 1 dure 6 heures, il faudrait donc produire 6 MWh supplémentaires. Le coût augmenterait donc $6\lambda_1 = 6c_B = 8,28$€.

2. Si on augmente la demande de $\delta > 0$ MW sur la période 2, tout en gardant la même base, on obtient : $x'_A = x_A + \delta = D_2 - U_B + \delta$. Pour rester dans la même base, il faut que : $x'_A \leq U_A$, c'est-à-dire $\delta \leq U_A + U_B - D_2 \Rightarrow \delta \leq 11500$ MW. La demande peut être **augmentée au maximum de 11500 MW sur la deuxième période sans altérer la statut optimal de la base**. Le surcoût associé serait de $\Delta C = \lambda \delta \Delta t = c_A \cdot \delta\, 3h$.

3. Même avec une nouvelle centrale B, la contrainte sur U_B serait saturée : la base ne change pas et la variation d'optimum est donc donnée par la variable duale associée à U_B : $\Delta C = \mu_{B,2} \Delta P_{B,max} = (c_B - c_A)\Delta P_{B,max}$. Pour une centrale B de 1750 MW sur la période 2, on aurait donc : $\Delta C = (1,38 - 1,50) \cdot 1750 \cdot 3h = -630$€. Le coût baisse car on substitue la production des centrales A par des centrales B à coût marginal plus faible.

4. Sur la première période, on retire l'équivalent d'une centrale du maximum de production : $U'_B = U_B - P_{max,N} = 15750$ MW. Or $D_1 = 15000 \leq U'_B$, donc la la capacité B reste suffisante pour couvrir la demande. La base et la technologie marginale restent les mêmes $\Rightarrow$ pas d'impact sur l'optimum de la période 1.

Sur la deuxième période, la contrainte U_B est saturée : $\Delta C = -\mu_{B,2} \Delta P_{B,max} = -(c_B - c_A)\Delta P_{B,max} = 0,12 \cdot 1750 \cdot 3h = +650$€. Le coût augmente car on substitue la production des centrales B par des centrales A à coût marginal plus important.

5. Le nouveau coût horaire de c'_B est donné par $c'_B = c_B + \Delta c_B$. La base est optimale tant que les contraintes sont respectées, notamment $\mu^*_B = c'_B - c_A > 0 \Rightarrow c'_B > c_A \Rightarrow \Delta c_B > c_A - c_B = 0.12€/MWh$. On peut donc **augmenter le coût horaire de B d'au plus 0,12 €/MWh sans modifier la base optimale**, c'est-à-dire tant que le coût marginal de B reste inférieur à celui de A.

1.5. Implémentation du modèle

Le modèle contient bien des variables discrètes : $n_{p,t}$.

D'après l'affichage des *logs* de la résolution du problème de minimisation par Gurobi (ci-dessous) :

- best objective ≈ 869
- best bound ≈ 869
- root relaxation ≈ 869
- heuristic solution ≈ 933

La solution optimale x^* est encadrée par des bornes inférieures et supérieures, qui varient successivement au cours de la résolution : $LB \leq x^* \leq UB$.

Valeur	Type de borne	Explication
Best objective	Supérieure (primale)	meilleur candidat pour le problème primal
Best bound	Inférieure (duale)	dans l'algorithme branch-and-bound, chaque noeud correspond à une version relaxée du problème : la solution associée est donc inférieure à la solution du problème MILP. La solution *best bound* correspond au maximum des solutions trouvées grâce au branch-and-bound.
Root relaxation	Inférieure (duale)	solution de la relaxation au premier noeud de branch-and-bound
Heuristic solution	Supérieure (primale)	première solution obtenue après l'application par le solveur d'heuristiques "simples"

Le modèle **ne possède pas la propriété d'unimodularité** car les coefficients de la matrice A ($Ax \leq b$) associés aux variables entières sont différents de -1 et 1 (par exemple : $P_{max,p}$ pour $x_{p,t} \leq n_{p,t} P_p^{max}$). Dans le contraire, on aurait pu résoudre directement la relaxation continue du problème (car LP=ILP en cas d'unimodularité) : ici, il faut donc bien considérer le problème MILP.

In [52]:
```
import model_1_5
m_1_5 = model_1_5.build_model(model_1_5.data_thermal, model_1_5.data_periods)
opt = optimizer.solve(m_1_5, tee=True,load_solutions=True)
print(opt)
print(f"C* = {pyo.value(m_1_5.obj)} €")
```

```
Read LP format model from file C:\Users\AACKER~1\AppData\Local\Temp\tmpl099_p71.pyomo.lp
Reading time = 0.04 seconds
x1: 50 rows, 30 columns, 90 nonzeros
Gurobi Optimizer version 12.0.3 build v12.0.3rc0 (win64 - Windows 11.0 (22631.2))

CPU model: 12th Gen Intel(R) Core(TM) i7-12700, instruction set [SSE2|AVX|AVX2]
Thread count: 12 physical cores, 20 logical processors, using up to 20 threads

Optimize a model with 50 rows, 30 columns and 90 nonzeros
Model fingerprint: 0x8adf59f5
Variable types: 15 continuous, 15 integer (0 binary)
Coefficient statistics:
  Matrix range     [1e+00, 4e+03]
  Objective range  [4e+00, 2e+01]
  Bounds range     [0e+00, 0e+00]
  RHS range        [5e+00, 4e+04]
Presolve removed 46 rows and 24 columns
Presolve time: 0.02s
Presolved: 4 rows, 6 columns, 12 nonzeros
Variable types: 0 continuous, 6 integer (0 binary)
Found heuristic solution: objective 958050.00000
Found heuristic solution: objective 947700.00000
Found heuristic solution: objective 933300.00000

Root relaxation: objective 8.694000e+05, 2 iterations, 0.00 seconds (0.00 work units)

    Nodes    |    Current Node    |     Objective Bounds      |     Work
 Expl Unexpl |  Obj  Depth IntInf | Incumbent    BestBd   Gap | It/Node Time

     0     0 869400.000    0    1 933300.000 869400.000  6.85%     -    0s
H    0     0                    873900.00000 869400.000  0.51%     -    0s
H    0     0                    869400.00000 869400.000  0.00%     -    0s
     0     0 869400.000    0    1 869400.000 869400.000  0.00%     -    0s

Explored 1 nodes (2 simplex iterations) in 0.02 seconds (0.00 work units)
Thread count was 20 (of 20 available processors)

Solution count 5: 869400 873900 933300 ... 958050

Optimal solution found (tolerance 1.00e-04)
Best objective 8.694000000000e+05, best bound 8.694000000000e+05, gap 0.0000%

Problem:
- Name: x1
  Lower bound: 869400.0
  Upper bound: 869400.0
  Number of objectives: 1
  Number of constraints: 50
  Number of variables: 30
  Number of binary variables: 0
  Number of integer variables: 15
  Number of continuous variables: 15
  Number of nonzeros: 90
  Sense: minimize
Solver:
- Status: ok
  Return code: 0
  Message: Model was solved to optimality (subject to tolerances), and an optimal solution is available.
  Termination condition: optimal
  Termination message: Model was solved to optimality (subject to tolerances), and an optimal solution is availabl
e.
  Wall time: 0.019999980926513672
  Error rc: 0
Solution:
- number of solutions: 0
  number of solutions displayed: 0

C* = 869400.0 €
```

1.6 Implémentation de la relaxation continue

In [30]:
```
m_1_5_relaxed = copy.deepcopy(m_1_5)
pyo.TransformationFactory("core.relax_integer_vars").apply_to(m_1_5_relaxed)
```

```
optimizer.solve(m_1_5_relaxed, tee=False)
print(f"C* = {pyo.value(m_1_5_relaxed.obj)} €")
```

```
C* = 869400.0 €
```

On a $C^*_{relax} = C^*$. Cela s'explique en accédant aux valeurs des variables (avec *model.display()*, non montré ici pour la lisibilité du document) : dans le cas de la relaxation continue, la variable $n_{p,t}$ n'est pas pénalisée dans la fonction objectif et n'est plus contrainte sur les valeurs qu'ele peut prendre : elle "suit" donc la variable $x_{p,t}$ optimale. Par exemple, $x_{A,15h-18h} = 22500$ MW $= n_{p,t} \cdot P_p^{max} = 11,25 \cdot 2000$ MW et ne modifie pas le résultat par rapport au problème MILP. De fait, comme $n_{p,t}$ n'apparaît pas dans la fonction objectif, la solution est la même dans les deux cas.

In [31]:
```
print("Période \t  Production A \t        Production B \t        Production C \t      Demande")
for t in m_1_5_relaxed.Periods:
    print(t, end="\t \t")
    for p in m_1_5_relaxed.Plants:
        print(f"{m_1_5_relaxed.prod[p, t].value} MW / {m_1_5_relaxed.Pmax[p]*m_1_5_relaxed.N[p]} MW", end="\t")
    print(f"{m_1_5_relaxed.Demand[t]} MW", end="")
    print("\n")
```

```
Période          Production A          Production B           Production C          Demande
0h-6h          0.0 MW / 24000 MW      15000.0 MW / 17500 MW   0.0 MW / 20000 MW      15000 MW

6h-9h          12500.0 MW / 24000 MW  17500.0 MW / 17500 MW   0.0 MW / 20000 MW      30000 MW

9h-15h         7500.0 MW / 24000 MW   17500.0 MW / 17500 MW   0.0 MW / 20000 MW      25000 MW

15h-18h        22500.0 MW / 24000 MW  17500.0 MW / 17500 MW   0.0 MW / 20000 MW      40000 MW

18h-24h        9500.0 MW / 24000 MW   17500.0 MW / 17500 MW   0.0 MW / 20000 MW      27000 MW
```

On observe numérique que la solution *merit-order* est optimale : la centrale B au coût marginal le plus faible est d'abord appelée pour satisfaire, puis, si la demande n'est pas satisfaite entièrement, la centrale A.

1.7. Reformulation de la relaxation continue

1. Reformulation sans contraintes redondantes

Sur la période 2, la relaxation continue du modèle (1.1), après suppression des variables/contraintes redondantes de (1.3), s'écrit :

$$\min c_A x_A + c_B x_B + c_C x_C \text{ s.c. : } \begin{cases} x_A + x_B + x_C = D_2 \\ 0 \leq x_p \leq U_p \quad \forall p \in A, B, C \end{cases}$$

C'est un PL à 3 variables bornées couplées par une seule contrainte d'égalité.

2. Variante de problème de sac-à-dos

On peut interpréter (P2) comme une variante de problème de sac-à-dos continu (fractionnaire) : chaque catégorie p est un « objet » pouvant être pris en quantité continue $x_p \in [0, U_p]$. Le « volume » est la puissance fournie, et la capacité du sac est fixée à D_2 (on doit remplir exactement D_2 MW). Le « coût par unité de volume » est c_p. La formulation générale du problème sac-à-dos continu est :

$$\min \sum_p c_p x_p \quad \text{s.c.} \quad \sum_p x_p = D_2, 0 \leq x_p \leq U_p.$$

L'algorithme optimal est celui du merit-order / sac-à-dos continu. On classe les catégories par coût marginal croissant et on remplit :

$$c_B < c_A < c_C \quad \Rightarrow \quad B \text{ puis } A \text{ puis } C.$$

On obtient donc la solution MO sur la période 2 : $x_B^\star = U_B = 17500$ MW, $x_A^\star = D_2 - U_B = 30000 - 17500 = 12500$ MW, $x_C^\star = 0$.

2. Coûts d'opération

2.1 Coûts de fonctionnement

Redéfinition de l'objectif : il change pour prendre en compte les coûts de base et les coûts variables :

$$\min \sum_{t \in T} \sum_{p \in A,B,C} [C_{base,p} n_{p,t} + C_{MWh,p}(x_{p,t} - P_{min,p} \cdot n_{p,t})] \cdot \Delta_t$$

In [32]:
```
import model_2_1
m_2_1 = model_2_1.build_model(model_2_1.data_thermal, model_2_1.data_periods)
optimizer.solve(m_2_1)
print(f"C* = {pyo.value(m_2_1.obj)} €")
```

```
C* = 978900.0 €
```

2.2. Coûts de démarrage

Le surcoût correspondant à l'allumage est :

$$\Delta C_{p,t} = max(0, n_{p,t} - n_{p,t-1}) \cdot C_{start,p}$$

La fonction max introduisant une non-linéarité, il faut donc introduire une variable qui compte le nombre de centrales allumées à chaque pas de temps. Comme seules les valeurs positives nous intéressent (pas de coût induit si des centrales sont éteintes), on limite la variable aux entiers positifs. A $t = 0$, le nombre de centrales démarrées correspond évidemment au nombre de centrales actives à $t = 0$. Formellement, on introduit donc la variable $n_{p,t}^{start}$ $\forall p \in \{A, B, C\}$, $t \in T$ avec les contraintes :

$$n_{p,t}^{start} \geq \begin{cases} n_{p,0} & \text{si } t = 0 \\ n_{p,t} - n_{p,t-1} & \text{sinon} \end{cases}$$

et $n_{p,t}^{start} \geq 0$.

L'objectif devient donc :

$$\min \sum_{t \in T} \sum_{p \in A,B,C} [C_{base,p} n_{p,t} + C_{MWh,p}(x_{p,t} - P_{min,p} \cdot n_{p,t}) + C_{start,p} \cdot n_{p,t}^{start}] \cdot \Delta_t$$

In [33]:
```
import model_2_2
m_2_2 = model_2_2.build_model(model_2_2.data_thermal, model_2_2.data_periods)
optimizer.solve(m_2_2)
print(f"C* = {pyo.value(m_2_2.obj)} €")
```

```
C* = 1014400.0 €
```

3. Réserve de puissance

L'ajout d'une réserve de puissance se traduit par la contrainte :

$$\sum_{p} (P_{max,p} \cdot n_{p,t} - x_{p,t}) \geq \alpha \cdot D_t \quad \forall t \in T$$

avec $\alpha = 0,15$ ici.

In [34]:
```
import model_3_1
m_3_1 = model_3_1.build_model(model_3_1.data_thermal, model_3_1.data_periods)
optimizer.solve(m_3_1)
print(f"C* = {pyo.value(m_3_1.obj)} €")
```

```
C* = 1015150.0 €
```

On constate qu'entre 15h et 18h, il est fait appel à 3000 MW de la centrale C pour satisfaire à la nouvelle exigence de réserve, ce qui n'était pas le cas auparavant.

4. Planification cyclique

La prise en compte de la planification cyclique revient à rendre "visible" pour le modèle le nombre de centrales de chaque technologie allumée à la dernière période de la veille. Le coût moindre obtenu dans ce cas provient de coûts d'allumage plus faibles, certaines centrales étant déjà allumées à la dernière période de la veille.

En termes mathématiques, cela revient à ajouter une nouvelle contrainte :

$$n_{p,t}^{start} \geq \begin{cases} n_{p,t} - n_{p,\max(T)} & \text{si } t = 0 \\ n_{p,t} - n_{p,t-1} & \text{sinon} \end{cases}$$

où $n_{p,\max(T)}$ correspond au nombre de centrales de la technologie p allumées à la dernière période de la veille.

En Python, on implémente d'abord le modèle non-cyclique duquel on extrait les valeurs de $n_{p,t}$ pour la dernière période, que l'on fournit ensuite en entrée au modèle cyclique.

In [35]:
```
import model_4_1
m_4_1 = model_4_1.build_model(model_4_1.data_thermal, model_4_1.data_periods, optimizer, cyc=True)
optimizer.solve(m_4_1)
print(f"C* = {pyo.value(m_4_1.obj)} €")
```

```
C* = 988540.0 €
```

5. Centrales hydroélectriques

5.1. Centrales hydroélectriques

L'ensemble des centrales est maintenant défini par $C = \{A, B, C, H1, H2\}$. On introduit des *blocks* en Pyomo pour faciliter l'implémentation et la future différenciation entre centrales hydroélectriques et thermiques.

In [36]:
```
import model_5_1
m_5_1 = model_5_1.build_model(model_5_1.data_thermal, model_5_1.data_hydro, model_5_1.data_periods, optimizer, cy
optimizer.solve(m_5_1)
print(f"C* = {pyo.value(m_5_1.obj)} €")
```

```
C* = 890260.0 €
```

5.2 Pompage

On introduit deux nouvelles variables correspondant au pompage :

- $f_{in,t}$:= flux d'eau pompé de la rivière vers le réservoir à la période t
- $f_{out,p,t}$:= flux d'eau tiré du réservoir pour l'alimentation de la centrale p à la période t

Ces deux variables sont soumises aux contraintes suivantes :

- $f_{out,p,t} = W_p \cdot n_{p,t} \cdot \Delta t$ (consommation en eau de la centrale p au temps t, avec W_p la consommation horaire d'eau de la centrale hydroélectrique) ;
- $\sum_{t \in T} f_{in,t} = \sum_{t \in T} \sum_{p \in P} f_{out,p,t}$ (compensation du pompage sur la journée)

Le pompage de l'eau induit une consommation électrique supplémentaire, qui n'est pas "gratuite" et vient s'ajouter à la demande initiale. La demande se définit donc maintenant comme :

- $d_t = D_t + \sum_{p \in H1,H2} f_{in,t} \cdot \dfrac{E_p}{\Delta t}\ [MW]$ où D_t est la demande par période telle que définie dans l'énoncé et $E_p[MWh/m]$ la consommation de la pompe pour élever le niveau d'un mètre.

In [37]:
```
import model_5_2
m_5_2 = model_5_2.build_model(model_5_2.data_thermal, model_5_2.data_hydro, model_5_2.data_periods, optimizer, cy
optimizer.solve(m_5_2)
print(f"C* = {pyo.value(m_5_2.obj)} €")
```

```
C* = 988540.0 €
```

5.3 Paliers de fonctionnement

On note $L = \{1, 2, 3, 4\}$ les paliers de fonctionnement.

On introduit la variable :

$$y_{p,l,t} = \begin{cases} 1 & \text{si la centrale hydroélectrique } p \text{ tourne au palier } l \text{ à la période } t \\ 0 & \text{sinon} \end{cases}$$

Cette variable est soumise à la la contrainte :

$$\sum_{l \in L} y_{p,l,t} \leq 1 \quad \forall p \in \{H1, H2\} \quad \forall t \in T$$

(la centrale ne peut fonctionner qu'à un seul palier à chaque pas de temps).

In [38]:
```
import model_5_3
m_5_3 = model_5_3.build_model(model_5_3.data_thermal, model_5_3.data_hydro, model_5_3.data_periods, optimizer, cy
optimizer.solve(m_5_3)
print(f"C* = {pyo.value(m_5_3.obj)} €")
```

```
C* = 987135.0 €
```

5.4. Exclusion pompage et génération hydro

On introduit une contrainte de type big-M :

$$f_{in,t} \leq M \cdot (1 - \sum_{p \in \{H1,H2\}} \sum_{l \in L} y_{p,l,t}) \quad \forall t \in T$$

On a bien $f_{in,t} = 0$ lorsqu'au moins une des centrales hydroélectriques produit.

Dans le cas extrême, le pompage compense sur une unique période l'eau qui a été consommée par les deux centrales tournant à plein au palier 4 toute la journée. On peut donc choisir $M = \sum_{p,t} W_{p,4} * \Delta t$

In [39]:
```
import model_5_4
m_5_4 = model_5_4.build_model(model_5_4.data_thermal, model_5_4.data_hydro, model_5_4.data_periods, optimizer, cy
optimizer.solve(m_5_4)
print(f"C* = {pyo.value(m_5_4.obj)} €")
```

```
C* = 987438.0 €
```

6. Désagrégation

6.1. Individualisation des centrales

La production des centrales thermiques est maintenant de la forme $x_{p,k,t}$, où k fait référence à la k-ième unité de la technologie p. Les contraintes sont modifiées en conséquence.

On introduit également une nouvelle variable binaire $z_{p,k,t} = \begin{cases} 1 & \text{si l'unité k est allumée au temps t} \\ 0 & \text{sinon} \end{cases}$. Elle remplace en quelque sorte la variable $n_{p,t}$, qui était propre à toute une technologie de centrale. On a ainsi la contrainte : $P_p^{min} z_{p,k,t} \leq x_{p,k,t} \leq P_p^{max} z_{p,k,t} \quad \forall p \forall k \quad \forall t \in T$. Notons que $n_{p,t} = \sum_k z_{p,k,t}$

In [40]:
```
import model_6_1
m_6_1 = model_6_1.build_model(model_6_1.data_thermal, model_6_1.data_hydro, model_6_1.data_periods, optimizer, cy
optimizer.solve(m_6_1)
print(f"C* = {pyo.value(m_6_1.obj)} €")
```

```
C* = 986771.0 €
```

6.2. Planification au pas horaire

- Avec contrainte d'exclusion pompage/hydro

In [41]:
```
import model_6_2
m_6_2 = model_6_2.build_model(model_6_2.data_thermal, model_6_2.data_hydro, model_6_2.data_periods, optimizer, cy
optimizer.solve(m_6_2)
print(f"C* = {pyo.value(m_6_2.obj)} €")
```

```
C* = 986775.0 €
```

- Sans contrainte d'exclusion pompage/hydro

In [42]:
```
import model_6_2_b
m_6_2_b = model_6_2_b.build_model(model_6_2_b.data_thermal, model_6_2_b.data_hydro, model_6_2_b.data_periods, opt
optimizer.solve(m_6_2_b)
print(f"C* = {pyo.value(m_6_2_b.obj)} €")
```

```
C* = 986180.0 €
```

6.3. Précision du profil de demande

- Avec contrainte d'exclusion pompage/hydro

In [43]:
```
import model_6_3
m_6_3 = model_6_3.build_model(model_6_3.data_thermal, model_6_3.data_hydro, model_6_3.data_periods, optimizer, cy
optimizer.solve(m_6_3)
print(f"C* = {pyo.value(m_6_3.obj)} €")
```

```
C* = 987655.0 €
```

- Sans contrainte d'exclusion pompage/hydro

In [44]:
```
import model_6_3_b
m_6_3_b = model_6_3_b.build_model(model_6_3_b.data_thermal, model_6_3_b.data_hydro, model_6_3_b.data_periods, opt
optimizer.solve(m_6_3_b)
print(f"C* = {pyo.value(m_6_3_b.obj)} €")
```

```
C* = 987130.0 €
```

6.4. Discrétisation à un pas de 2 heures

In [45]:
```
import model_6_4
m_6_4 = model_6_4.build_model(model_6_4.data_thermal, model_6_4.data_hydro, model_6_4.data_periods, optimizer, cy
optimizer.solve(m_6_4)
print(f"C* = {pyo.value(m_6_4.obj)} €")
```

```
C* = 989800.0 €
```

7. Disponibilité variable

7.1. Maintenance nocturne des centrales A

Dans le premier cas, la contrainte correspondante est :

$$n_{A,t} \leq 10 \quad \forall t \in [[18;24]] \cup [[0;6]]$$

Dans le second cas, la contrainte correspondante est :

$$x_{A,k,t} = 0 \quad \forall k \in \{1,2\} \quad \forall t \in [[18;24]] \cup [[0;6]]$$

Il n'y a pas de raison *a priori* pour que la fonction objectif prenne des valeurs différentes à l'optimum. Comme chaque centrale A a des coûts de démarrage, dans le cas 1, le solveur "choisira" deux uniques centrales à arrêter durant la nuit, ce qui est équivalent au cas 2. Si, dans le cas 1, le solveur mettait alternativement deux centrales différentes à l'arrêt au temps t pour satisfaire à la contrainte, alors cela induirait des coûts de démarrage supplémentaires pour les deux centrales qui avaient été éteintes en $t-1$, et donc éloignerait l'objectif de l'optimum.

In [46]:
```
import model_7_1
m_7_1 = model_7_1.build_model(model_7_1.data_thermal, model_7_1.data_hydro, model_7_1.data_periods, optimizer, cy
optimizer.solve(m_7_1)
print(f"C* = {pyo.value(m_7_1.obj)} €")
```

```
C* = 1003900.0 €
```

8. Contraintes de rampe

Note : à partir d'ici, les valeurs de C^* obtenues diffèrent de celles indiquées dans l'énoncé ; je n'ai pas réussi à trouver la cause de cet écart.

8.1. Rampe croissante

La contrainte s'écrit : $x_{p,k,t+1} - x_{p,k,t} \leq R_p^{up} \quad \forall t \in T \quad \forall p \in \{A, B, C\}$ avec $R_1^{up} = 400$ MW, $R_2^{up} = 600$ MW et $R_3^{up} = 800$ MW.

En cas de baisse de production ($x_{t+1} < x_t$), la contrainte de rampe croissante n'est pas restrictive.

In [47]:
```
import model_8_1
m_8_1 = model_8_1.build_model(model_8_1.data_thermal, model_8_1.data_hydro, model_8_1.data_periods, optimizer, cy
optimizer.solve(m_8_1)
print(f"C* = {pyo.value(m_8_1.obj)} €")
```

```
C* = 1034505.0 €
```

8.2. Rampe de démarrage

On sépare la contrainte en deux :

1. La contrainte sur la rampe se transforme en : $x_{p,k,t+1} - x_{p,k,t} \leq R_p^{up} + M(2 - z_{p,k,t+1} - z_{p,k,t})$. Il est facile de voir que si la centrale n'est pas active sur une des deux périodes, la contrainte devient inactive du fait du terme relatif au big-M. Dans le cas extrême, la variation de la puissance est de l'ordre de la puissance maximum de la centrale, on peut donc grossièrement choisir $M = P_p^{max}$.

2. La contrainte sur le démarrage s'écrit : $x_{p,k,t+1} - x_{p,k,t} \leq R_p^{start} + M(1 - z_{p,k,t+1} + z_{p,k,t})$. Le terme relatif au big-M est nul uniquement lorsque $z_{p,k,t} = 0$ et $z_{p,k,t+1} = 1$, c'est-à-dire lorsque la centrale s'allume. Là aussi, on peut choisir $M = P_p^{max}$.

In [48]:
```
import model_8_2
m_8_2 = model_8_2.build_model(model_8_2.data_thermal, model_8_2.data_hydro, model_8_2.data_periods, optimizer, cy
optimizer.solve(m_8_2)
print(f"C* = {pyo.value(m_8_2.obj)} €")
```

```
C* = 989352.0 €
```

8.3. Rampe décroissante et d'arrêt

1. La première contrainte s'écrit à l'aide d'un big-M : $x_{p,k,t} - x_{p,k,t+1} \leq R_p^{down} + M(2 - z_{p,k,t} - z_{p,k,t+1})$. Comme précédemment, on note que si la centrale n'est pas active sur une des deux périodes, la contrainte devient inactive du fait du terme relatif au big-M.

2. La seconde contrainte s'écrit de manière similaire : $x_{p,k,t} - x_{p,k,t+1} \leq R_p^{stop} + M(1 - z_{p,k,t} + z_{p,k,t+1})$. Comme précédemment, le terme relatif au big-M est nul uniquement lorsque $z_{p,k,t} = 1$ et $z_{p,k,t+1} = 0$, c'est-à-dire lorsque la centrale s'éteint. Là aussi, on peut choisir $M = P_p^{max}$.

In [49]:
```
import model_8_3
m_8_3 = model_8_3.build_model(model_8_3.data_thermal, model_8_3.data_hydro, model_8_3.data_periods, optimizer, cy
optimizer.solve(m_8_3)
print(f"C* = {pyo.value(m_8_3.obj)} €")
```

```
C* = 989485.0 €
```

9. Prévention de l'usure

9.1. Durée minimale d'activité

On introduit $s_{p,k,t} = \begin{cases} 1 & \text{si la centrale (p, k) démarre au temps t} \\ 0 & \text{sinon} \end{cases}$. Sa valeur est pilotée par l'introduction d'une contrainte : $s_{p,k,t} \geq z_{p,k,t} - z_{p,k,t-1}$ (pour rappel : $z_{p,k,t} = 1$ si la centrale (p, k) fonctionne au temps t, 0 sinon).

La contrainte sur la durée minimale d'activité s'écrit :

$$\sum_{\tau=t}^{t+L-1} z_{p,k,\tau} \geq L s_{p,k,t} \quad \forall p \in \{A, B, C\}, \quad \forall k \in K_p, \quad \forall t \in \{0, 1, \ldots, 23 - L\}$$

Ici, on fixe $L = 8h$. Si la centrale s'allume au temps t, $s_{p,k,t} = 1$ et la contrainte impose que la somme des variables d'état de la centrale sur les L heures suivantes doit être supérieure ou égale à L, c'est-à-dire qu'elle doit être allumée au moins L heures sur les L heures suivantes : on obtient le comportement recherché.

In [50]:
```
import model_9_1
m_9_1 = model_9_1.build_model(model_9_1.data_thermal, model_9_1.data_hydro, model_9_1.data_periods, optimizer, cy
optimizer.solve(m_9_1)
print(f"C* = {pyo.value(m_9_1.obj)} €")
```

```
C* = 987719.0 €
```

9.2. Durée minimale d'arrêt

Similairement à la question précédente, on introduit $e_{p,k,t} = \begin{cases} 1 & \text{si la centrale (p, k) s'arrête au temps t} \\ 0 & \text{sinon} \end{cases}$. Sa valeur est pilotée par l'introduction d'une contrainte : $e_{p,k,t} \geq z_{p,k,t-1} - z_{p,k,t}$.

La contrainte sur la durée minimale d'arrêt s'écrit de manière relativement similaire à la contrainte de la question précédente :

$$\sum_{\tau=t+1}^{t+L} z_{p,k,\tau} \leq L(1 - e_{p,k,t}) \quad \forall p \in \{A, B, C\}, \quad \forall k \in K_p, \quad \forall t \in \{0, 1, \ldots, 23 - L\}$$

On garde la même valeur de $L = 8h$. Si la centrale s'éteint au temps t, $1 - e_{p,k,t} = 0$: la contrainte impose que la somme des variables d'état de la centrale sur les L heures suivantes doit être inférieure ou égale à 0, c'est-à-dire qu'elle doit être allumée moins de 0 heure sur les L heures suivantes : on obtient le comportement recherché. A l'inverse, si $e_{p,k,t} = 0$, la centrale peut fonctionner L heures sur les L heures suivantes : la contrainte n'est pas active.

In [51]:
```
import model_9_2
m_9_2 = model_9_2.build_model(model_9_2.data_thermal, model_9_2.data_hydro, model_9_2.data_periods, optimizer, cy
optimizer.solve(m_9_2)
print(f"C* = {pyo.value(m_9_2.obj)} €")
```

```
C* = 987718.9999999999 €
```